THE OPEN-SOURCE AUDIT

A Project Report for OSS NGMC Course

Open-Source Software Vityarthi

Student Name	Nrishan Jyoti Das
Registration Number	24BCE11026
Slot	A24
Date of Submission	31st March 2026
Chosen Software	Git (Version Control, GPL v2)
Course	Open-Source Software | VIT Bhopal

1. Introduction

This is my report for the OSS Project at VIT Bhopal. For this project, I had to pick a real open-source software project, audit its workings, and back up my analysis with five shell scripts to show I know my way around the Linux command line.

I picked Git on purpose. It's not just some open-source tool — it's the backbone of pretty much every software project out there. Git's origin story is very interesting: Linus Torvalds, the guy who created Linux, ended up building Git in just two weeks. He did it out of pure frustration because the version control system they'd been using yanked away its free license. That rush job? It turned into the world's most used version control system. That says so much about what open source is all about: necessity, collaboration, and sharing.

Git uses the GPL v2 license, just like the Linux kernel. It's fast, distributed, and built to let lots of people work together without stepping on each other's toes. By digging into why Git exists, how it's managed, what its license actually allows, and how it stacks up against closed alternatives, you get a clearer picture of the values that drive today's software world. This goes way beyond just coding — it's about how people build, share, and trust in the tools they use every day.

2. Part A — Origin and Philosophy

2.1 A1: The Origin Story of Git

To really get why Git exists, you have to rewind to how things worked before 2005 for the Linux kernel team—and the particular mess that pushed Linus Torvalds to write his own tool from scratch.

The BitKeeper Era (2002–2005)

Back in 2002, the Linux kernel had already exploded into a massive, global project. Hundreds of contributors sent code patches every week. Trying to manage all those changes by trading tar archives and patches over email just didn't scale. Eventually, the team switched to using BitKeeper, a commercial, distributed version control system from BitMover, run by Larry McVoy. The catch? BitKeeper was closed-source. The Linux team got a special free license, as long as they didn't try to reverse-engineer the protocol or build a competing tool.It was never a comfortable solution. On one hand, here was the crown jewel of free software being managed with a proprietary tool—which didn't sit well with plenty of developers. Still, the tool worked well and honestly, the alternatives were worse.

The Breaking Point (April 2005)

Everything blew up in April 2005. Andrew Tridgell, best known for creating Samba, started poking at BitKeeper's protocol and wrote a script to read and write BitKeeper repositories. He argued this was classic open-source research. Larry McVoy didn't see it that way. Suddenly, BitMover yanked access for the whole Linux kernel team. The most important open-source project on Earth was completely stuck overnight. They had no version control at all.Linus Torvalds looked at existing tools, like CVS and Subversion, but neither worked for the kernel. Both were centralized—which meant every single code change had to go through one server. For a project as big and spread out as the kernel, that was a dead end. Linus also hated CVS's design on a fundamental level.

Two Weeks in April 2005

He didn't waste time. In early April 2005, Linus got to work. This wasn't a "weekend side project"—it was emergency surgery. He started on April 3rd. By April 7th, Git was already tracking its own development. By April 16th, the Linux kernel was using Git. So in roughly two weeks, he built the foundation of the modern world's version control.

Git's design came from the pain points of everything that failed before it. It had to be fast—really fast. The Linux kernel juggles an outrageous number of files and commits. It had to be distributed, so every developer got a full copy of project history. And it had to make data integrity non-negotiable: every commit and file is identified by a SHA-1 hash, so you can prove it hasn't been tampered with. Importantly, it had to be free—licensed under GPL v2, so another BitKeeper debacle couldn't happen.

A few months later, Linus handed off Git's stewardship to Junio C. Hamano, who still leads the project. Linus built it, plugged the gap, and passed it on. It was never meant as his personal project; it's true community infrastructure.

From Kernel Tool to Universal Standard

Git's wider explosion in popularity didn't take long, especially once GitHub launched in 2008. Before that, Git was powerful but awkward for small teams, pretty much tailored to kernel-style workflows. GitHub polished Git's rough edges and made things like pull requests and

code review easier. Suddenly, Git wasn't just for open source projects—it became the backbone of the entire software world.

Now, over 93% of developers use Git, according to Stack Overflow's survey. It's taught everywhere. The tool Linus whipped up in a panic became the core way developers track code changes worldwide. That's Git's origin: frustration, necessity, urgency, and a license that promised the whole community couldn't be locked out again.

2.2 A2: The License — What GPL v2 Actually Says

Git uses the GNU General Public License version 2 (GPL v2). You'll find the full license text inside Git's source at git/COPYING. But you need more than a marketing summary or a sticker that says "open source"—the details matter.

The Four Freedoms of Free Software

The Free Software Foundation sets out four core freedoms, mostly thanks to Richard Stallman, and these build the backbone for licenses like the GPL:

- Freedom 0: Run the program for any reason, no strings attached.
- Freedom 1: Study how the program works (which means you get the source code) and change it.
- Freedom 2: Share copies with others.
- Freedom 3: Share your modified versions so everyone benefits.

Does Git's license deliver all four? Absolutely. Download Git, use it for anything, read every line, share it, or make your own variant and publish it. That's exactly why Linus picked GPL v2—he'd just watched what happens when your tools don't guarantee those rights. BitKeeper didn't give you Freedom 1 or Freedom 3, which made it possible for BitMover to yank access.

Can a Company Modify Git and Sell It Closed-Source?

They can't. GPL v2's main guardrail is "copyleft": if you distribute anything based on GPL v2 code, you must share your changes under the same license and give out the source code. You can't legally take this kind of work private.

This is a big difference from permissive licenses like MIT or Apache 2.0. There, you can take code, make hidden changes, sell it, and never even mention your alterations. With GPL v2, the sharing logic stays glued to the project; critics call it "viral," fans say it protects the community.Git can't go proprietary. Sure, companies use it to build commercial services (think GitHub or GitLab), but those services don't ship modified Git binaries. They use Git as-is in their stacks. If you use GPL-licensed software in your infrastructure, that's fine—the copyleft rules only kick in if you distribute modified software.

If I were starting a project now, I'd pick GPL for anything I want the community to always access—including infrastructure. That's why Linus made that choice; he didn't want a repeat of the BitKeeper "locked out" problem. But if my main goal was for my library to be used as widely as possible—even in closed, commercial software—I'd probably go with MIT. For original work, I lean GPL, but I get the MIT argument.

Free Beer vs Free Freedom

If I were starting a project now, I'd pick GPL for anything I want the community to always access—including infrastructure. That's why Linus made that choice; he didn't want a repeat of the BitKeeper "locked out" problem. But if my main goal was for my library to be used as widely as possible—even in closed, commercial software—I'd probably go with MIT. For original work, I lean GPL, but I get the MIT argument.

One last bit: "Free beer" versus "free freedom." That's Stallman's phrase—free as in speech, not just as in beer. Lots of software is free to download, but locked up tight. You can't peek at how they work, can't improve them, and can't share modified versions. Skype, for example, is free to download but 100% proprietary. Git is both: free to grab, and genuinely free in what you can do with it. GPL v2 is about protecting your liberty, not just your wallet.

That's the difference.

2.3 A3: The Ethics of Open Source

Should All Software Be Open Source?

That's tricky. It's easy to just say yes, but real life's not so simple.

There are solid arguments for universal open source. Having more eyes on code means bugs and security problems get found and fixed faster, which really matters for anything that handles personal, financial, or public-facing data. Open source also lets people and companies collaborate in a way that pumps out better results than almost any one organisation could manage. And when important software is open, everyone has access—there's less risk of lock-in or some company pulling the rug out from under you.

But "open by default" isn't always the answer. Not all software can safely be public. Security tools, the stuff running inside pacemakers, financial trading bots—publicly revealing every detail could help the wrong people. Plus, developers need to get paid, and open source doesn't always keep the lights on, even if some companies have made it work (Red Hat, Canonical). There's also the "freeloader" problem: a small team can build something huge, and then watch as giant corporations make millions off their work without giving anything back.

So what's my take? The critical, shared tools—the ones software depends on, like operating systems or version control—these should definitely be open source. Making them proprietary just brings back all the problems Git was created to escape. But it's fine if not all software is open. Ask who gets hurt if this is closed vs. open, and that's usually your answer.?

Is It Ethical to Build a Business on Free Community Labour?

This is the question that the open-source world is still wrestling with in 2026. Companies like Google, Meta, and Microsoft use Linux extensively — Linux runs almost all of Google's servers, powers Android, and underlies most of the infrastructure of the modern internet. They have contributed back to varying degrees, but the economics are asymmetric: the companies extract billions in value and contribute back code and developers, but the core

maintainers of many critical open-source projects are still individuals working part-time, often burning out.

Red Hat built its entire business model on selling support and services around GPL software. There is a legitimate argument that this is precisely what the GPL was designed to allow: you can make money from free software through services, not by locking the code. Red Hat contributes significantly to many projects it profits from. That is a more defensible model.

The harder question is the company that takes an open-source library, uses it in a product generating hundreds of millions in revenue, and contributes almost nothing back. This is technically permitted under permissive licences like MIT. Whether it is ethical depends on your view of reciprocity. I think there is a reasonable expectation that companies that build on community labour should invest back, not legally, but as a matter of professional ethics. The open-source community has noticed this asymmetry — the emergence of source-available licences and the Business Source Licence are direct responses to it.

Standing on the Shoulders of Giants

Here's the big picture. Most of the tools I use as a developer exist because people were willing to share: Python, Git, GCC, Linux, basically the whole internet stack. No one invents everything from scratch these days—you build on what already exists. This is how real innovation happens; open code lets ideas compound and spread faster than any closed model. Some people claim this stifles originality. I don't buy it. The model delivers a different kind of motivation—fame, reputation, and the sheer joy of contributing—but it absolutely works. We just need better ways to fund the work so those maintainers don't burn out.

3. Part B — Linux Footprint

Git lives on a Linux system differently from most software because it is not a daemon or a service — it is a command-line tool that runs as the invoking user. Understanding its Linux footprint means understanding where it installs, how it is managed, and what its permissions story looks like.

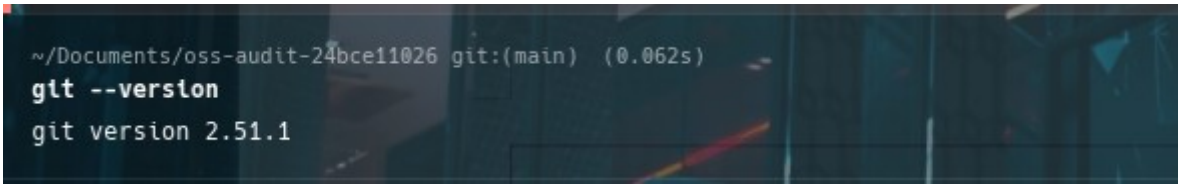
Installation Method

Git is available through the package managers of all major Linux distributions, which means it can be installed with a single command and managed (updated, removed) the same way:

```
# Debian / Ubuntu
sudo apt update && sudo apt install git

# Fedora / RHEL / CentOS
sudo dnf install git

# Arch Linux
sudo pacman -S git
```



Key Directories and Files

Path	Purpose
/usr/bin/git	Main Git binary. Confirm with: which git
/usr/lib/git-core/	Git sub-commands and helper executables (git-remote, git-fetch, etc.)
/usr/share/git-core/	Templates, contrib scripts, shell completions
/usr/share/doc/git/	Documentation and changelogs
~/.gitconfig	Per-user configuration (name, email, aliases, preferences)
/etc/gitconfig	System-wide configuration (applies to all users on the machine)
<repo>/.git/	Per-repository Git data: objects, refs, HEAD, config, hooks

User and Group

Unlike servers (nginx runs as www-data, MySQL as mysql), Git doesn't have a dedicated system user. Whenever you run git, it runs as you—the logged-in user. That's intentional; Git is a developer's tool, not something humming quietly in the background. All the files it touches in a repo—the ones hiding in .git/—belong to whoever ran the command.

This design matters when a system is shared between users. In fact, Git 2.35.2 closed a security hole (CVE-2022-24765) where running Git inside a directory owned by someone else could let attackers escalate privileges with custom git hooks. Open source meant anyone could spot the bug, submit a fix, and get it patched quickly.

Service Management

You won't manage Git itself with systemctl because it's not a service. There is a git-daemon included, which lets you serve up repositories over git://, and that one can run as a service if you want. Otherwise, it doesn't run unless you tell it to.

Update Model

Updates and security fixes flow in through the usual Linux channels—your package manager handles everything. The Git project's maintainer (Junio C. Hamano) tags new releases, and distributors like Ubuntu or Fedora package them up and release security patches through their update process. If you run sudo apt upgrade, and there's a new Git version, you get the fix. That's one way open source wins: anyone can audit code, security issues get fixed by the community, and updates come straight through your operating system's trusted infrastructure.

4. Part C — The FOSS Ecosystem

Dependencies

Git doesn't do everything alone. When you build it from source, it leans on quite a few open-source libraries and tools:

- Libcurl: handles cloning and pushing over HTTP/HTTPS
- openssl / gnutls: lets Git verify TLS certificates when it's talking to servers
- zlib: for data compression (Git stores all objects compressed)
- libpcre2: for regular expression support in git grep
- Perl: required for several Git utility scripts (like git-svn, git-send-email)
- Python: used in some contrib tools and git-p4 (Perforce bridge)
- gettext: for internationalisation and translation support
- expat: parses XML, necessary for git-http-push

All these are open-source too, usually under licenses compatible with GPL v2. In the FOSS world, this is just how things work — software constantly builds on top of other software, and the openness just layers up along the way.

What Git Enabled and Inspired

The way Git works — distributed, fast, and all about content-addressed storage — ended up launching a whole universe of services and tools. Here's a snapshot:

Project	What it is
GitHub (2008)	The world's largest code hosting platform. Acquired by Microsoft in 2018 for $7.5 billion. Popularised pull requests and social coding.
GitLab	Open-core DevOps platform with self-hosted and cloud options. The main open-source alternative to GitHub.
Gitea	Lightweight, self-hosted Git service written in Go. Very popular in privacy-conscious and embedded deployments.
Bitbucket	Atlassian's Git hosting, popular in enterprise environments integrated with Jira.
libgit2	A portable C library implementing the Git core methods. Powers GitHub's backend, GitHub Desktop, and many language bindings.
JGit	A pure Java implementation of Git, used inside Eclipse and Android Studio.
go-git	A pure Go Git implementation used by many DevOps automation tools.
Gerrit	Code review tool built on top of Git, used by Android and Google's internal projects.

Relation to the Web and the LAMP Stack

Git isn't technically a piece of the classic LAMP stack (Linux, Apache, MySQL, PHP) — it's not serving web pages. But if you work with web applications, you're living in Git's world

anyway. Every PHP deployment, every Apache tweak, every MySQL database migration is tracked in Git, especially if your team means business. Commits set off CI/CD pipelines. “Infrastructure as code” tools (Terraform, Ansible, Helm) keep all their stuff version-controlled in Git, too.

If LAMP is the engine that drives web apps, Git is the shipping department and warehouse. Nothing rolls out, gets upgraded, tested, or backed up without Git these days

Community and Governance

Git is run in a style that comes straight from Linus Torvalds, the way he manages the Linux kernel: flat, by email, and with little bureaucracy. You want to contribute? You send a patch to git@vger.kernel.org. Forget about pull requests on GitHub — the real work happens via mailing list.

- Junio C. Hamano has been leading the project since 2005. He reviews patches, sets the pace for releases, and writes the release notes.
- No foundation, no board, no corporate overlords. It keeps things lean, but it’s also a little risky — Junio’s workload is the chokepoint.
- The Git Merge conference brings the community together once a year. They’ve done it in New York, Barcelona, Brussels, Stockholm — all over.
- For security issues, there’s a dedicated mailing list (git-security@googlegroups.com) where vulnerabilities get discussed privately before they go public.

5. Part D — Open Source vs Proprietary

If you’re comparing Git to the closest “closed source” alternatives, the main standouts are Perforce Helix Core and Microsoft’s Azure DevOps version control. (SVN is technically open source too.) Perforce is the heavyweight in enterprise and gaming, so let's compare Git with that.

Dimension	Git (Open Source, GPL v2)	Perforce Helix Core (Proprietary)
Cost	Free forever. No licence fee for any number of users or repositories.	Free up to 5 users and 20 workspaces; paid licence required beyond that. Enterprise deployments cost thousands per year.
Security (Code Audit)	Full source available on kernel.org. Community worldwide can find and report vulnerabilities. CVEs are patched quickly and transparently.	Closed source. Trust the vendor for security. Security issues depend on Perforce’s own internal audit processes.
Support and Reliability	Community-supported through Stack Overflow, mailing lists, and extensive documentation. Commercial support available from GitHub Enterprise, GitLab, etc.	Dedicated paid support contracts. Guaranteed SLAs for enterprise customers. Phone support available.
Freedom to Modify	Full. GPL v2 allows anyone to fork, modify, and redistribute Git (under the same licence).	None. Source code is not available. You use it as the vendor provides it.
Community vs Corporate Control	Governed by an individual maintainer (Junio Hamano) and community	Roadmap entirely controlled by Perforce Software, Inc. Features

	contributors via public mailing list. No single company controls the roadmap.	prioritised by enterprise customer demand.
Handling Large Binary Files	Not ideal natively; requires Git LFS (Large File Storage), an extension. Learning curve for large asset pipelines.	Designed for large binary files from the ground up. Used extensively in game development (Unreal Engine, game assets).
Learning Curve	Steep initial learning curve, especially the staging area and rebase model. Widely taught globally now.	Different mental model (changelists vs commits) but well-documented for enterprise users.

Conclusion and Verdict

Would I tell a real-world team to use Git? Absolutely. In practice, at this point, it's not even a choice — not using Git is what needs an explanation, not the other way around. It's free, it's fast, and everyone on the team always has a complete backup of everything that's ever happened in the repo, thanks to it being distributed. Plus, the ecosystem of CI/CD tools, code review platforms, and IDEs just makes life easier. The only real use case where I'd look at Perforce instead is a huge game studio drowning in big binary files, where Git (even with LFS) can struggle.

Would I contribute to Git? I'd like to, someday. The process, though, is a bit intimidating — you have to use the mailing list, and the reviews are slow but thorough. No pull requests or point-and-click. But Git's such an important tool, and learning how to contribute to it would be a mark of real growth as a developer. I'm not ready today. But it's on my to-do list

6. Shell Scripts

Here are five shell scripts, all in the oss-audit-24bce11026 GitHub repo. I'll explain what each does, how it works, and the shell concepts behind them.

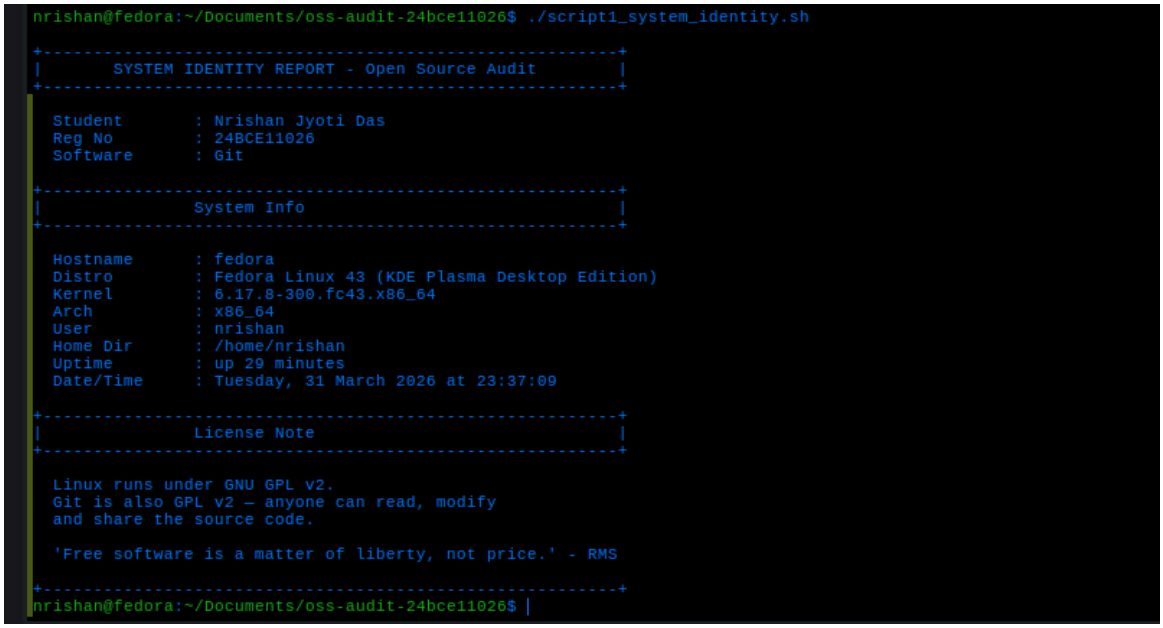
6.1 Script 1 — System Identity Report

What It Does

This script spits out a formatted welcome screen showing system info, who I am, which software I picked, and a line about GPL v2. It looks like those banners admins see on login, just tailored toward open-source auditing.

Shell Concepts Used

- Variables like STUDENT_NAME, KERNEL, DISTRO_NAME, etc.
- Command substitution, so stuff like uname -r and whoami dumps into shell variables.
- if statement guards reading /etc/os-release (not every system has it).
- Uses grep and cut to pluck PRETTY_NAME from /etc/os-release.
- echo prints a nice box; printf is used for formatting in other scripts.



6.2 Script 2 — FOSS Package Inspector

What It Does

This one checks if Git is installed, using dpkg or rpm depending on your distro. If Git's found, it reports the version and maintainer, then prints a one-liner about it (or another FOSS package, depending on your setting). It also shows how shell scripts adapt based on the Linux distribution.

Shell Concepts Used

- command -v: checks whether a command exists without running it, used to detect whether the system uses dpkg or rpm.
- if-then-else: nested conditionals handle the dpkg path, rpm path, and unknown systems.
- case statement: matches the package name against known FOSS projects and prints a relevant philosophy note for each.
- Pipe with grep: dpkg -s "$TARGET_PKG" | grep -E '^(Version|Maintainer|Homepage):' filters specific fields from verbose package info.
- Output redirection to /dev/null: &>/dev/null suppresses output during existence checks.

6.3 Script 3 — Disk and Permission Auditor

What It Does

This one loops through some critical system directories and reports on their permissions, who owns them, their group, and how big they are. It also checks the permissions on your git config files and explains why that matters.

Shell Concepts Used

- Array declaration: SYS_DIRS=("/etc" "/var/log" ...) stores the list of directories as a Bash array.
- for loop: for CURR_DIR in "${SYS_DIRS[@]}" iterates through the array. The [@] syntax expands all elements correctly even if paths have spaces.
- Directory test: [-d "$CURR_DIR"] checks if the path exists and is a directory before attempting to read it.
- awk for field extraction: ls -ld piped to awk '{print $1, $3, $4}' extracts permissions, owner, and group columns.
- du -sh for human-readable sizes: reports directory sizes in KB/MB/GB without overwhelming precision.
- printf for aligned output: %-18s and %-30s format strings pad fields to fixed widths, creating a clean table appearance.

```
nrishan@fedora:~/Documents/oss-audit-24bce11026$ ./script3_disk_auditor.sh

+-------------------------------------------------------+
|          Disk and Permission Auditor                  |
+-------------------------------------------------------+

  Path                | Perms / Owner / Group          | Size
  ---------------------------------------------------------------
  /etc                | drwxr-xr-x. root root          | 38M
  /var/log            | drwxr-xr-x. root root          | 1.5G
```

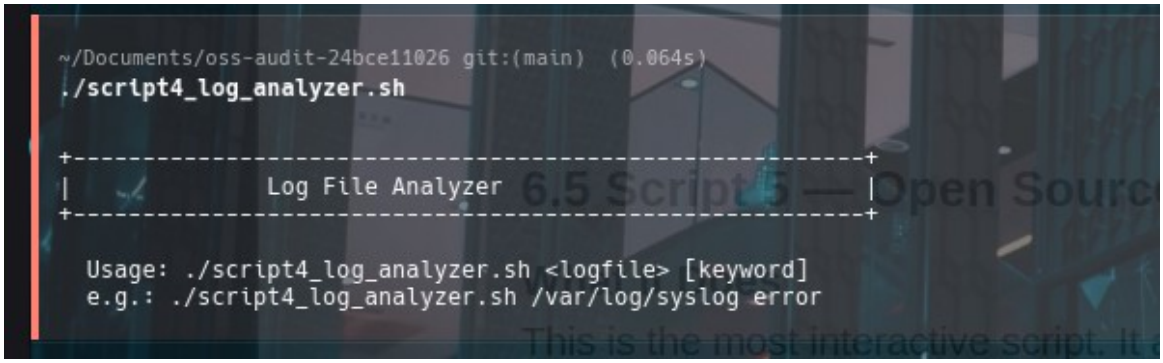
6.4 Script 4 — Log File Analyzer

What It Does

This script takes a log file path and an optional keyword (default is 'error'). It counts lines matching the keyword (case-insensitive), shows the last five matches, and asks you for another file up to a set number of retries if the first one isn't there.

Shell Concepts Used

- Positional parameters: $1 receives the log file path, $2 receives the keyword. ${2:-"error"} uses Bash parameter expansion to set a default if $2 is not provided.
- while true / break loop: implements retry logic for missing files, asking the user for an alternative path up to MAX_RETRY times before giving up.
- while IFS= read -r LINE loop: the standard Bash pattern for reading a file line by line without word splitting or backslash interpretation.
- Arithmetic counter: COUNT=$((COUNT + 1)) increments the match counter using POSIX arithmetic expansion.
- grep -i for case-insensitive matching inside a pipeline: the result feeds back into the same variable space.
- Tail of matches: grep -i "$SEARCH_WORD" "$LOG_PATH" | tail -5 extracts only the last five matching lines for the summary.



6.5 Script 5 — Open Source Manifesto Generator

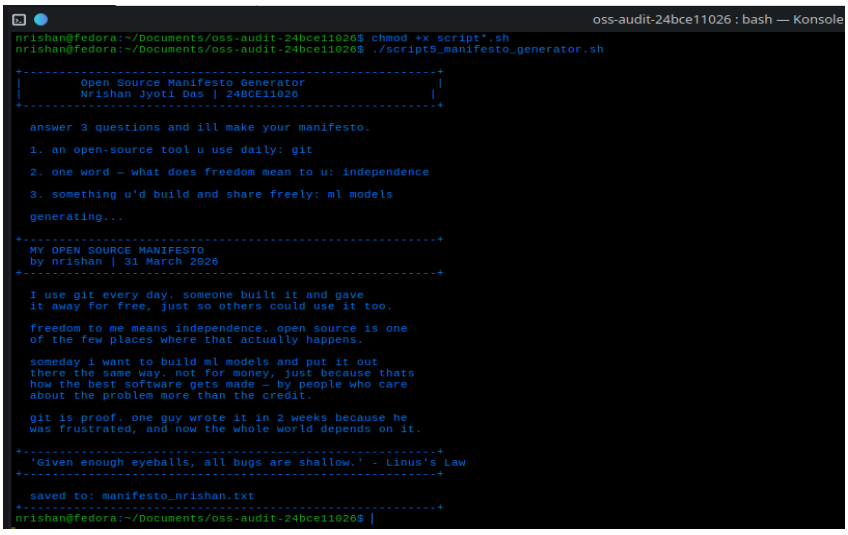
What It Does

This is the most interactive script. It asks you three questions about your open-source experience, takes your answers, and builds a .txt “manifesto” named after your current user. The script then shows it in the terminal.

Shell Concepts Used

- read -p: displays a prompt and waits for user input, storing the response in a named variable (DAILY_TOOL, FREEDOM_WORD, SHARE_IDEA).
- String interpolation in echo: variables are embedded directly in quoted strings, with Bash expanding them at runtime.
- File writing with > (overwrite) and >> (append): the first echo uses > to create/overwrite the output file, subsequent lines use >> to append, building the document paragraph by paragraph.
- $(whoami) for dynamic file naming: the output filename is manifesto_$(whoami).txt, so each user gets their own file.
- $(date) for timestamp: embeds the generation date in the manifesto header.
- Alias concept (in comments): the script demonstrates that shell aliases like alias gs='git status' are single-session shortcuts; for persistence, they belong in ~/.bashrc.

Output-



Conclusion

This project encouraged me to examine Git from a perspective I had never considered before. Despite using it daily—and relying on it for every project I have developed, including a production-level blockchain application—I had never taken the time to think about its origins, its creators, or the practical implications of its license.

The story behind Git is particularly striking. After a proprietary company revoked access to a tool used by the Linux kernel team, Linus Torvalds created a superior alternative in just two weeks. He released it under a copyleft license to ensure that no one could restrict access in the same way again, and later entrusted its maintenance to Junio Hamano, who has continued to lead its development for decades. This is not simply a story about software—it highlights what can happen when someone with deep technical insight chooses to solve a problem without being driven by commercial motives.

Reading and understanding the GPL v2 license was also more insightful than expected. The concept of copyleft, where the license extends to derivative works, represents more than a legal requirement—it reflects a broader philosophy about preserving freedom in software. It emphasizes that once freedoms are granted, they should remain protected and accessible to others.

Overall, this project gave me a greater appreciation for the individuals who have built the foundational tools that modern software development depends on. It also helped me better understand what it truly means to contribute to such infrastructure, rather than simply using it.

"Given enough eyeballs, all bugs are shallow."

Linus's Law